

Symulacja ruchu trzech ciał z ładunkami

Michał Królikowski, Gabryiel Czółnowski

June 1, 2026

1 Opis fizyki

Problem polega na opisie ruchu trzech ciał które przyciągają się ale też i odpychają się siłą elektrostatyczną. Program bazuje na tych wzorach:

$$\mathbf{F}_{ij} = k_e \frac{q_i q_j}{|\mathbf{r}_i - \mathbf{r}_j|^3} (\mathbf{r}_i - \mathbf{r}_j)$$

Figure 1: Wektorowa postać prawa Coulomba

$$\mathbf{F}_i = m_i \mathbf{a}_i = m_i \frac{d^2 \mathbf{r}_i}{dt^2}$$

Figure 2: Druga zasada Newtona

$$\frac{d^2 \mathbf{r}_i}{dt^2} = \sum_{j \neq i} \frac{k_e q_i q_j}{m_i} \frac{(\mathbf{r}_i - \mathbf{r}_j)}{|\mathbf{r}_i - \mathbf{r}_j|^3}$$

Figure 3: Równanie ruchu układu wielu ładunków punktowych

$$\text{Distance}^3 = (|\mathbf{r}_i - \mathbf{r}_j|^2 + \epsilon^2)^{3/2}$$

Figure 4: Wzór na odległość z wygładzeniem potencjalnym

2 Kod

Kod oblicza co chwile nowe przyspieszenie, porusza zgodnie ciała, oraz wyświetla je.

```
def coulomb_derivatives(t, state, m, q): 1 usage

    r = state[:9].reshape((3, 3))
    v = state[9:]

    a = np.zeros((3, 3))

    for i in range(3):
        for j in range(3):
            if i != j:
                dr = r[i] - r[j]
                dist_sq = np.sum(dr**2) + epsilon**2
                dist = np.sqrt(dist_sq)

                a[i] -= (k_e * q[i] * -q[j] / m[i]) * (dr / dist**3)

    return np.concatenate((v, a.flatten()))
```

Figure 5: Kod opisujący ciało

```

def init(): 1 usage
    for p, t in zip(points, trails):
        p.set_data([], [])
        p.set_3d_properties([])
        t.set_data([], [])
        t.set_3d_properties([])
    return points + trails

def update(frame): 1 usage

    for i in range(3):

        points[i].set_data([r_sol[i, 0, frame]], [r_sol[i, 1, frame]])
        points[i].set_3d_properties([r_sol[i, 2, frame]])

        start_idx = max(0, frame - 100)
        trails[i].set_data(r_sol[i, 0, start_idx:frame], r_sol[i, 1, start_idx:frame])
        trails[i].set_3d_properties(r_sol[i, 2, start_idx:frame])

    ax.view_init(elev=20., azim=frame/5.0)

    return points + trails

ani = FuncAnimation(fig, update, frames=len(t_eval), init_func=init, blit=False, interval=20)
plt.show()

```

Figure 6: Kod aktualizujący i obliczający zmiany co klatkę

3D 3-Body Coulomb Simulation

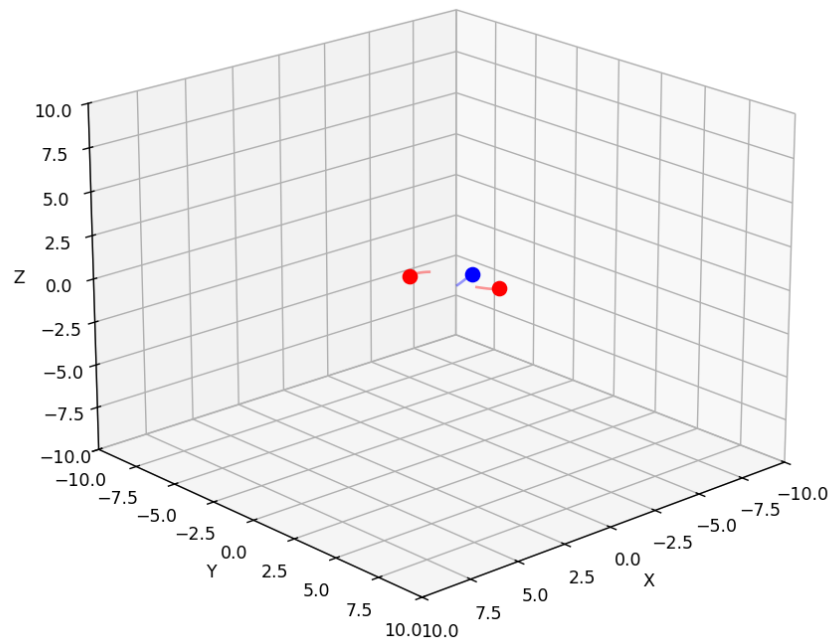


Figure 7: Okienko symulacji